Análisis de observabilidad con Arize Phoenix

Orquestador multi-agente de vulnerabilidades (LangGraph) · Supervisor + Investigador + Analista
09/11/2026 · Repositorio: github.com/fhofman/SpecializedMulti-agentOrchestrator

Resumen ejecutivo

Se generaron **11 trazas** en el proyecto *vuln-orchestrator-traffic* de Phoenix: 5 consultas complejas sobre los 4 ecosistemas (Maven, npm, PyPI, Packagist), 2 escenarios de fallo provocados y 4 re-ejecuciones de verificación después de corregir los problemas detectados. En total: **752 spans**, 99 llamadas al LLM (gpt-4o-mini-2024-07-18), 59 llamadas a herramientas, **201.161 tokens** y un costo estimado de **US$ 0,0332**.

- **Span de mayor latencia:** una generación del LLM (no la recuperación). Fue *ChatOpenAI* dentro del Analista en Q1-java-spring: **16,46 s**. En tiempo de reloj, la generación LLM explica el **94,5%** de la latencia y la recuperación (OSV.dev + Tavily) solo el **5,4%**.
- **Proceso de razonamiento más largo:** el Investigador en Q5-polyglot: 3 iteraciones ReAct, 6 llamadas a herramientas, **20.377 tokens** (19.769 de entrada, 608 de salida), US$ 0,0026.
- **Fallos observados en las trazas:** (1) una herramienta caída tumbaba todo el grafo; (2) consultas fuera de alcance terminaban con un falso "no hace falta parchear"; y, descubierto navegando las trazas, (3) el Investigador usaba nombres de repos de GitHub en PyPI y obtenía 0 vulnerabilidades (falso negativo).
- **Los tres se corrigieron y se verificaron con nuevas trazas** (sufijo *-fix*): sin caídas, sin falsos negativos y hasta 84% menos tokens en el caso fuera de alcance.

1. Instrumentación

El sistema usa **OpenInference** (*LangChainInstrumentor*) sobre OpenTelemetry. El exportador OTLP/HTTP apunta al servidor local de Phoenix (*http://localhost:6006/v1/traces*), configurado en *tracing.py*. Cada ejecución del grafo es una traza; cada nodo (supervisor, researcher, analyst, synthesizer), cada llamada al LLM y cada herramienta es un span. Las trazas llevan *session_id* = id del escenario y metadata filtrable (escenario, tipo, consulta).

Sobre la "búsqueda RAG": en este sistema la recuperación de información no es sobre una base vectorial sino sobre **fuentes externas**, como permite la consigna: *query_osv* consulta la base OSV.dev y *web_search* consulta Tavily. Son los spans de recuperación analizados en este documento.

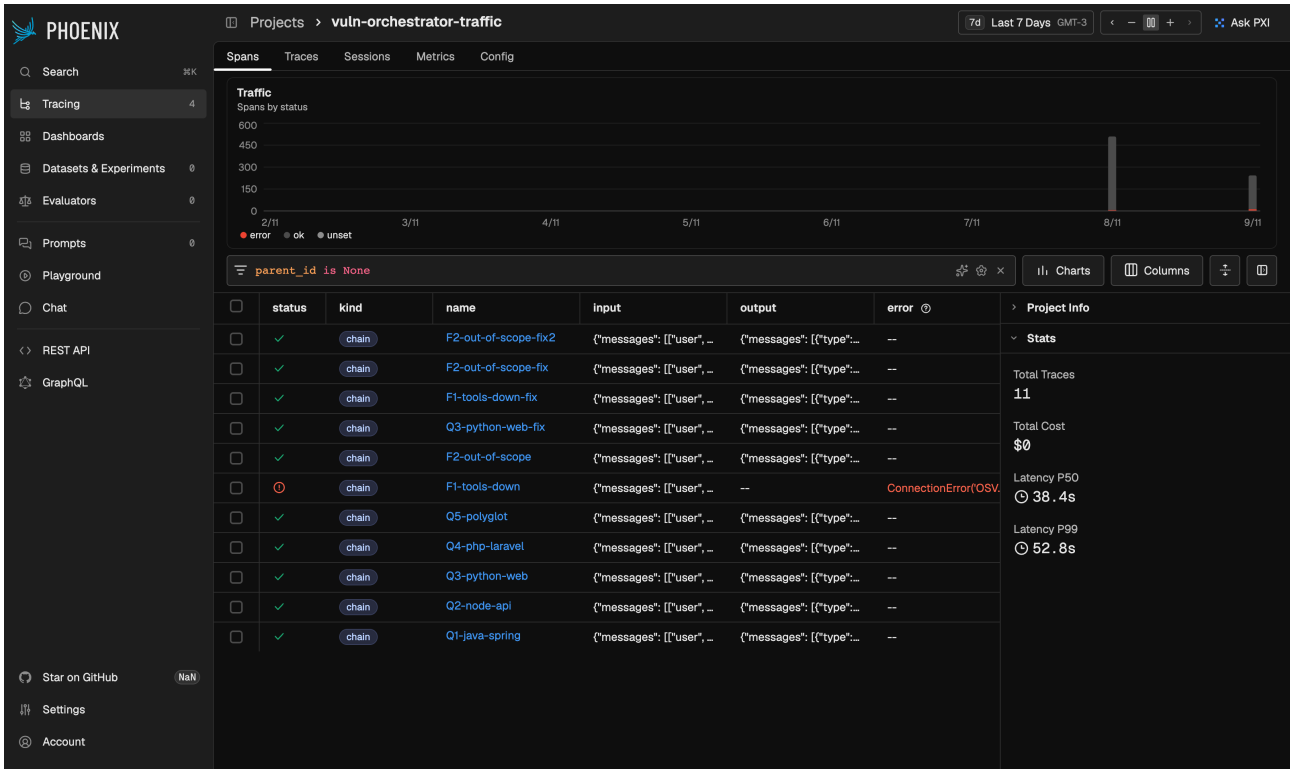
```
uv run phoenix serve
uv run python scripts/generate_traffic.py
uv run python scripts/analyze_traces.py && uv run python scripts/build_report.py
```

2. Tráfico generado

Las 5 consultas obligan a colaborar a los tres agentes: el supervisor delega, el Investigador usa *query_osv* (una vez por paquete, en paralelo) y *web_search*, el Analista usa *rank_dependencies* (cálculo CVSS en Python) y el supervisor valida con su rúbrica antes de cerrar.

Escenario	Qué pide	Recorrido del supervisor	Estado	Latencia	Tokens
Q1-java-spring	spring-web, jackson-databind, struts2 (Maven): riesgo de RCE	researcher -> analyst -> FINISH	OK	49,5 s	19.035
Q2-node-api	express, jsonwebtoken, axios, minimist (npm) + explotación activa	researcher -> analyst -> analyst -> FINISH	OK	53,3 s	32.012
Q3-python-web	requests, pyyaml, flask, jinja2, pillow (PyPI)	researcher -> analyst -> analyst -> analyst -> FINISH	OK	38,0 s	16.854

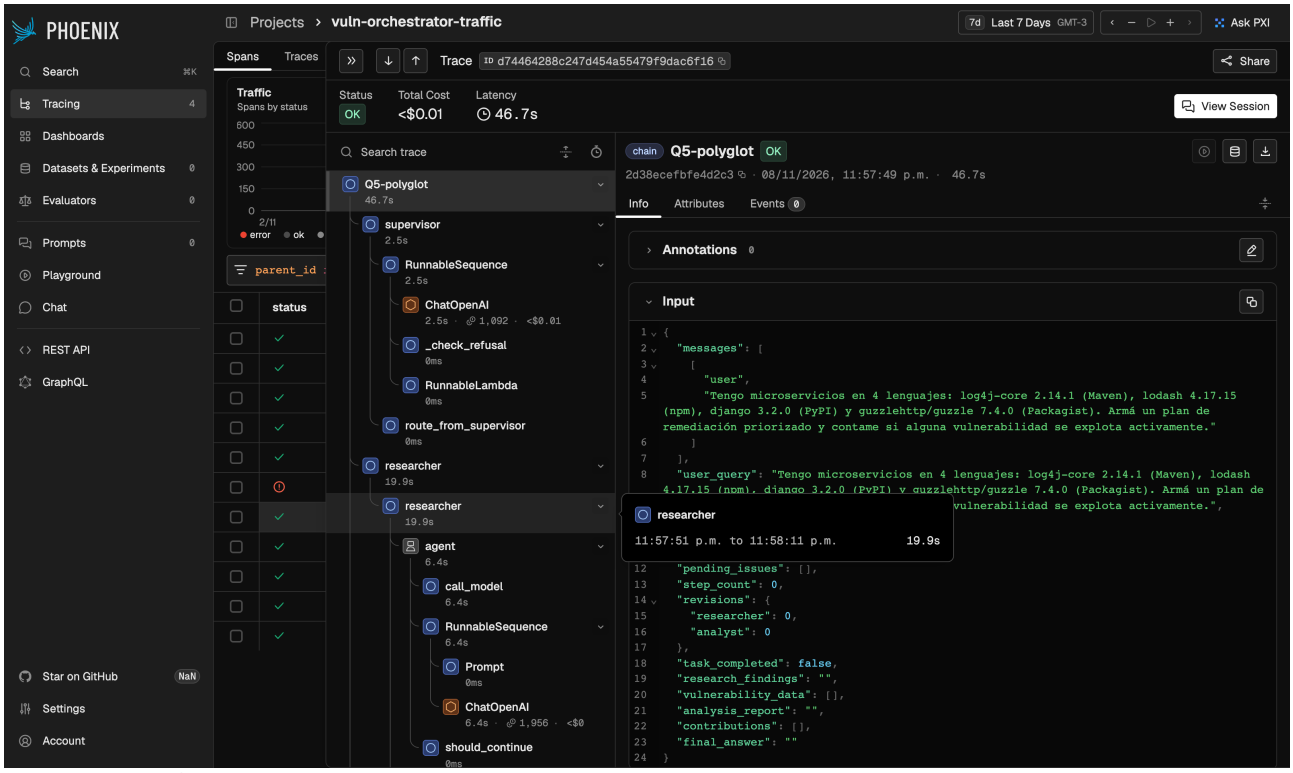
Escenario	Qué pide	Recorrido del supervisor	Estado	Latencia	Tokens
Q4-php-laravel	laravel, symfony/http-kernel, phpmailer, twig (Packagist)	researcher -> analyst -> FINISH	OK	43,1 s	18.770
Q5-polyglot	log4j, lodash, django, guzzle (4 ecosistemas) + explotación activa	researcher -> analyst -> FINISH	OK	46,7 s	31.208
F1-tools-down	**Fallo provocado:** OSV.dev desconectado + Tavily con API key inválida	researcher -> **excepción**	ERROR	4,6 s	2.815
F2-out-of-scope	**Fallo provocado:** firmware de router y Windows 10 (fuera de alcance)	researcher -> analyst -> analyst -> analyst -> FINISH	OK	40,0 s	22.389



Phoenix · lista de trazas del proyecto (filtro *parent_id is None*). F1-tools-down aparece con estado de error y el mensaje ConnectionError. Las trazas -fix son las re-ejecuciones posteriores a las correcciones.

Navegación de una traza completa

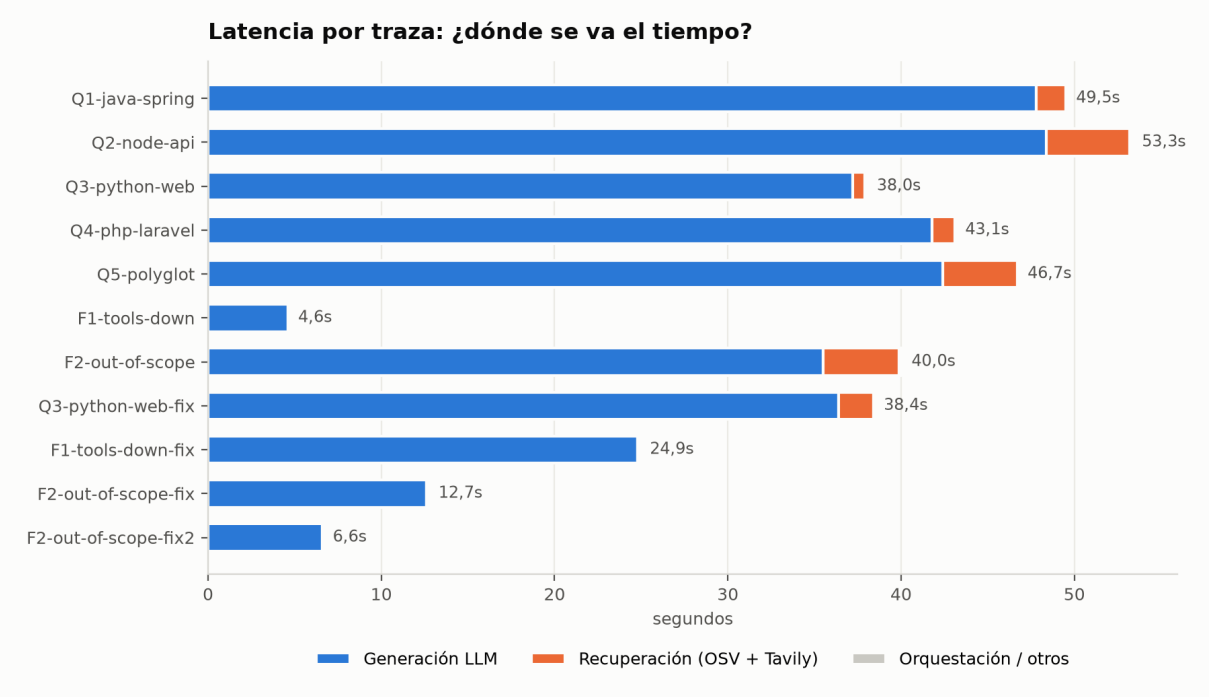
Al abrir una traza, Phoenix muestra el árbol de spans con la latencia de cada uno. En Q5 se ve la delegación: *supervisor* (2,5 s) decide, *researcher* (19,9 s) ejecuta su agente ReAct (*agent -> call_model -> ChatOpenAI*, luego *tools -> query_osv*), y así sucesivamente. Cada span LLM muestra tokens y costo; cada span de tool, sus argumentos y su resultado.



Phoenix · traza Q5-polyglot (46,7 s): árbol de spans de la delegación supervisor -> researcher -> analyst -> synthesizer.

3. ¿Cuál fue el span de mayor latencia?

Respuesta: fue una generación del LLM, no la recuperación. El span más lento de todo el tráfico es *ChatOpenAI* dentro del **Analista en Q1-java-spring**: **16,46 s** (2.299 tokens de entrada, 469 de salida), redactando la tabla de ranking y el plan de acción. **Los 10 spans hoja más lentos son todos llamadas al LLM**; ninguna herramienta entra en el top 10.



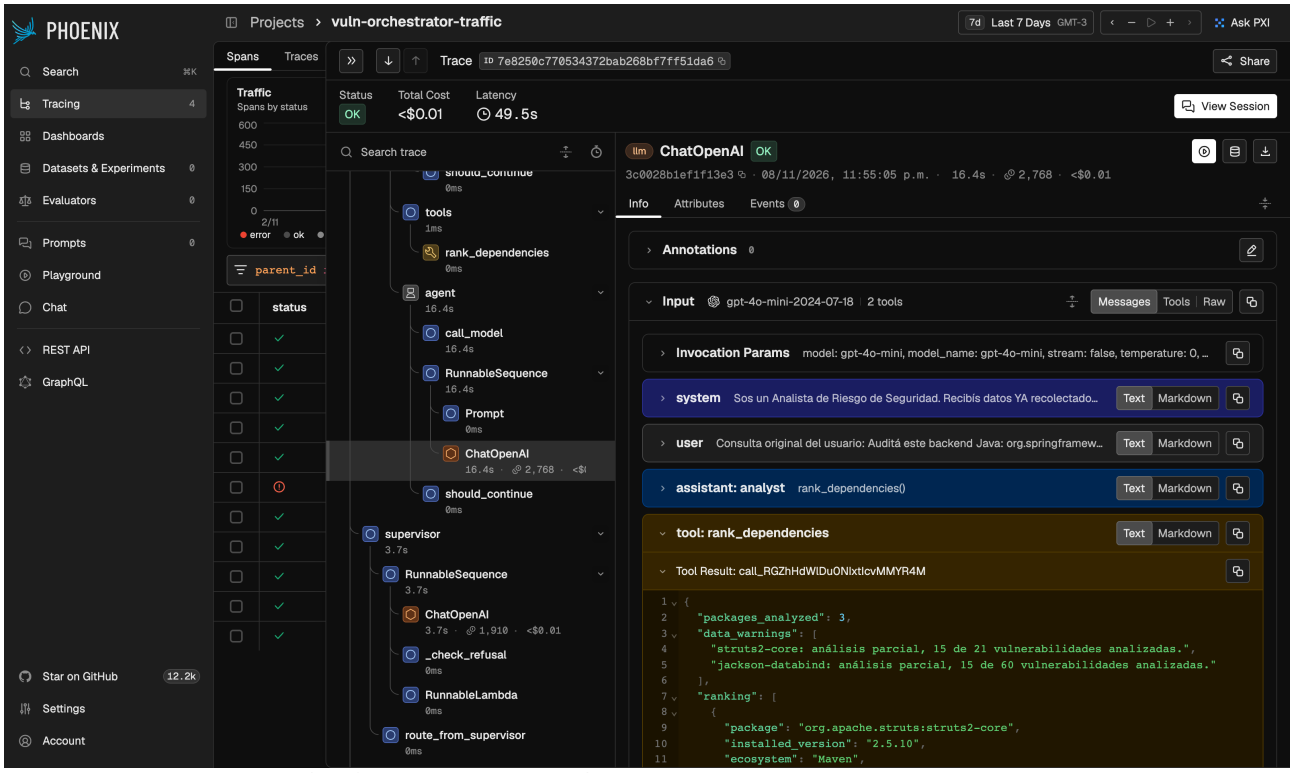
Latencia de cada traza dividida en tiempo de generación LLM y tiempo de recuperación (OSV + Tavily), medido como tiempo de reloj: las consultas a OSV corren en paralelo, así que se usa la unión de intervalos y no la suma.

Métrica	Generación LLM	Recuperación (OSV + Tavily)
Latencia media por llamada	3,41 s	1,29 s
Mediana (p50)	2,31 s	1,06 s
Máximo	16,46 s	3,63 s
Parte del tiempo total (reloj)	94,5%	5,4%

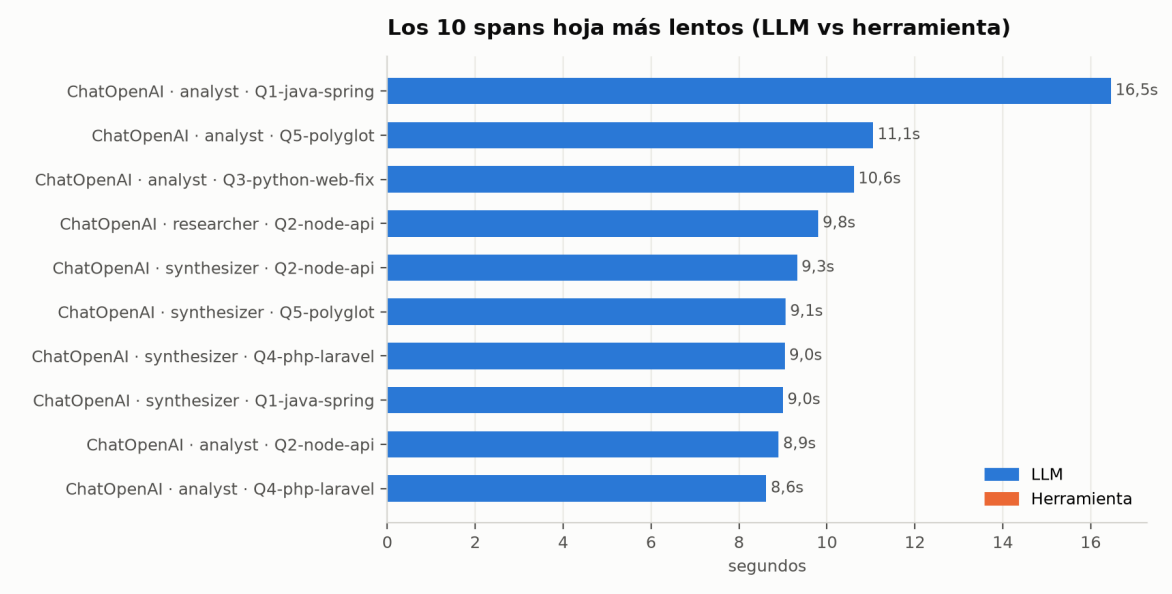
Por qué: la latencia sigue a los tokens de SALIDA

La correlación entre la latencia de un span LLM y sus tokens de salida es **0,87**; con los tokens de entrada es solo **0,30**. Por eso los spans más lentos son los del Analista y la Síntesis (escriben tablas y planes de ~450-700 tokens), aunque el Investigador procese prompts mucho más grandes. Generar texto token a token es lo caro en tiempo; leer contexto es relativamente barato.

#	Span	Agente	Escenario	Latencia	Tokens entrada	Tokens salida
1	ChatOpenAI (LLM)	analyst	Q1-java-spring	16,46 s	2.299	469
2	ChatOpenAI (LLM)	analyst	Q5-polyglot	11,05 s	2.675	532
3	ChatOpenAI (LLM)	analyst	Q3-python-web-fix	10,61 s	2.968	697
4	ChatOpenAI (LLM)	researcher	Q2-node-api	9,80 s	7.163	416
5	ChatOpenAI (LLM)	synthesizer	Q2-node-api	9,32 s	1.185	597
6	ChatOpenAI (LLM)	synthesizer	Q5-polyglot	9,06 s	1.228	644



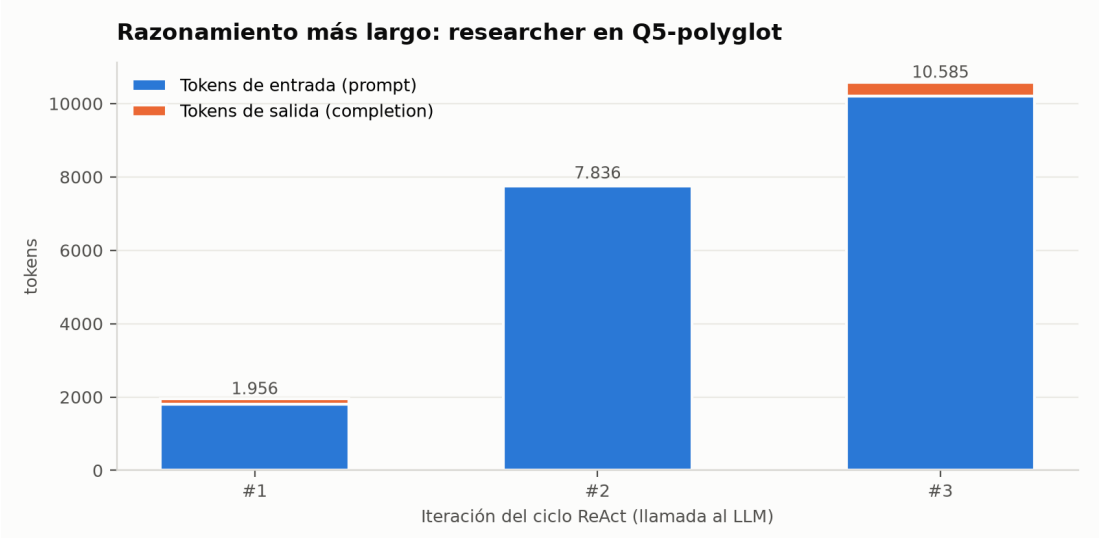
Phoenix · el span más lento (ChatOpenAI del Analista en Q1, 16,4 s). Justo arriba se ve la herramienta rank_dependencies: 0 ms. El cálculo determinístico es instantáneo; el tiempo lo consume la redacción del LLM.



Los 10 spans hoja más lentos: todos son generaciones del LLM.

4. Análisis de costo (tokens)

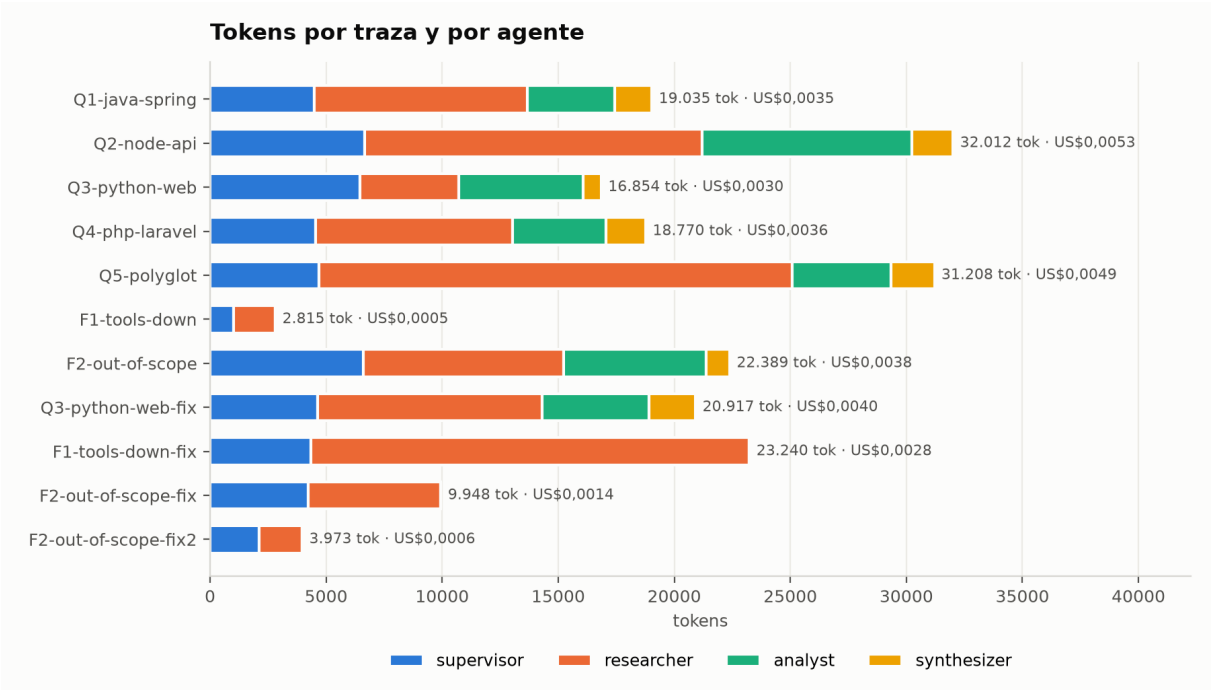
Respuesta: el proceso de razonamiento más largo fue el del Investigador en Q5-polyglot, con **20.377 tokens** (19.769 de entrada + 608 de salida) en 3 iteraciones del ciclo ReAct y 6 llamadas a herramientas, durante 20,0 s. Costo estimado: US$ 0,0026. Esa sola ejecución representa el 65% de los tokens de su traza.



Tokens por iteración del razonamiento más largo. El prompt crece en cada vuelta porque el agente ReAct re-envía todo su historial, incluidos los JSON devueltos por OSV.

El crecimiento es el patrón clave: iteración 1 = 1.809 tokens de entrada; iteración 2 = 7.760 (ya incluye las respuestas de OSV de 4 paquetes); iteración 3 = 10.200. El **97%** de los tokens del razonamiento son de entrada. En el Investigador, el costo lo determina el tamaño de los resultados de las herramientas, no lo que escribe.

Escenario	Agente	Iter.	Tools	Tokens entrada	Tokens salida	Total	Costo
Q5-polyglot	researcher	3	6	19.769	608	20.377	US$ 0,0026
Q2-node-api	researcher	3	6	13.909	621	14.530	US$ 0,0020
Q3-python-web-fix	researcher	2	5	9.212	462	9.674	US$ 0,0015
Q1-java-spring	researcher	2	3	8.725	467	9.192	US$ 0,0015
F2-out-of-scope	researcher	3	4	8.241	400	8.641	US$ 0,0012



Tokens por traza y por agente, con el costo estimado de cada traza. El Investigador es el agente que más tokens consume en casi todas las trazas.

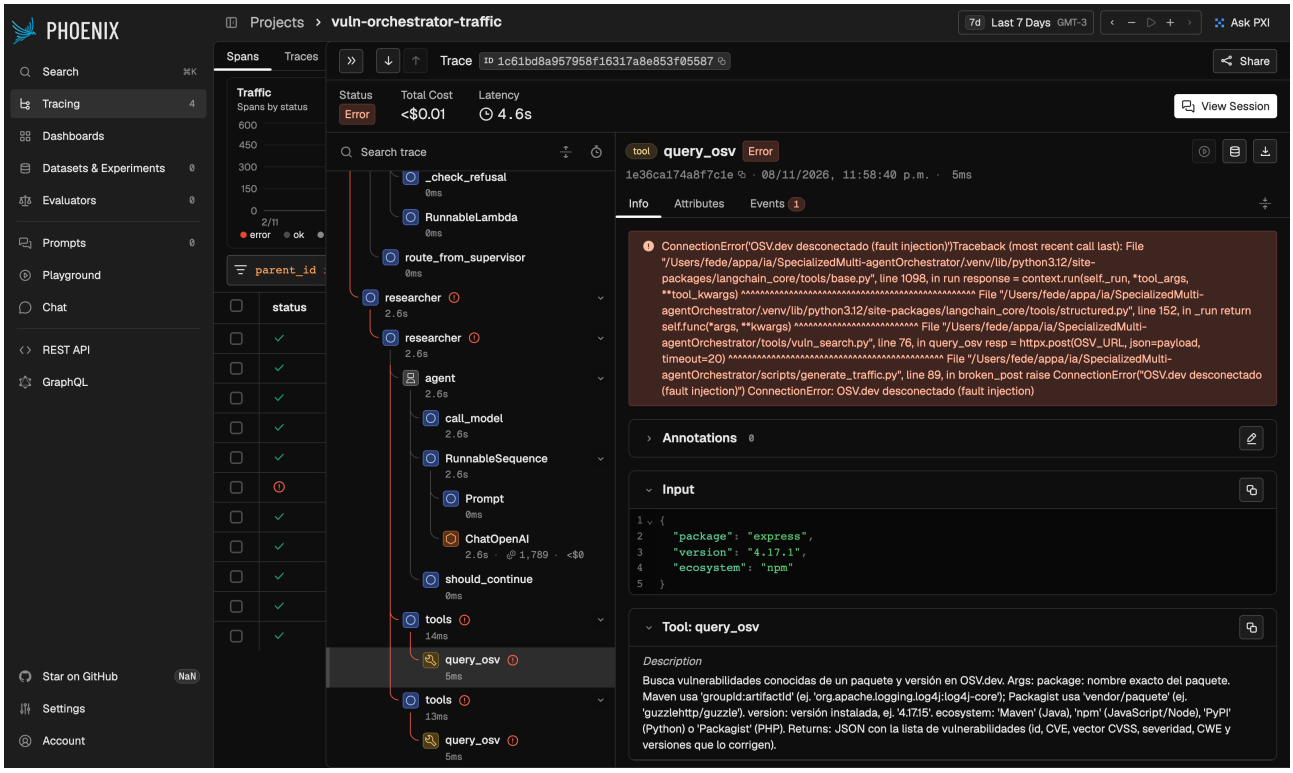
Precios usados (lista de OpenAI para gpt-4o-mini, USD por millón de tokens): entrada 0.15, entrada cacheada 0.075, salida 0.6. El panel de Phoenix muestra *Total Cost $0* porque no tiene precio configurado para el modelo versionado *gpt-4o-mini-2024-07-18*; por eso el costo se calculó a partir de los tokens registrados en cada span.

5. Fallos provocados: cómo se registran en la traza

F1 · Herramienta desconectada

Con *fault injection* en el script de tráfico, la llamada HTTP a OSV.dev lanza *ConnectionError* y Tavily recibe una API key inválida.

- **Cómo lo registra Phoenix:** el span TOOL *query_osv* queda en estado **Error**, con un evento de excepción que guarda el traceback completo (archivo y línea, incluida la línea de la inyección). El error **se propaga hacia arriba**, marcando en rojo *tools -> researcher -> researcher -> F1-tools-down* (7 spans en error). La traza raíz queda en **Error** con latencia de 4,6 s.
- **Lo que reveló:** el *ToolNode* de LangGraph, por defecto, **vuelve a lanzar** cualquier excepción que no sea de validación. Una sola herramienta caída tumbaba el grafo completo y el usuario no recibía respuesta.
- **Matiz:** Tavily con key inválida **no** lanza excepción: devuelve *{'error': '401 Unauthorized'}* como resultado normal. En la traza ese span figura como OK. Es una falla *silenciosa* que solo se ve leyendo el output del span.



Phoenix · span query_osv en error dentro de F1-tools-down: excepción con traceback, argumentos de entrada y la cadena de spans padres marcada en rojo.

F2 · Pedido fuera de alcance

Se pidió analizar el firmware de un router y Windows 10, que no son dependencias de Maven/npm/PyPI/Packagist. La traza no tiene errores técnicos (todos los spans en OK), pero muestra un **error de comportamiento**: el Analista corrió 3 veces sobre datos vacíos (22.389 tokens, 40,0 s) y la respuesta final afirmó que *"no es necesario aplicar parches"*, una conclusión peligrosa.

Q3 · Falso negativo descubierto navegando las trazas

Q3 respondió que requests 2.25.0, pyyaml 5.3, flask 1.0 y otros paquetes "no tienen vulnerabilidades". Al abrir los spans *query_osv* en Phoenix, los argumentos mostraron la causa: el LLM consultó *psf/requests*,

pallets/flask y *yaml/pyyaml* (nombres de repos de GitHub) en el ecosistema PyPI; OSV devolvió *total_found: 0* para cada uno. Sin la observabilidad de los inputs y outputs de cada tool, este error habría sido invisible: la traza estaba "verde".

6. Correcciones aplicadas y verificación con nuevas trazas

Hallazgo	Corrección	Antes	Después (traza -fix)
F1: una tool caída tumba el grafo	*ToolNode(handle_tool_errors=True)* en ambos agentes: la excepción vuelve al LLM como mensaje de error	Excepción, sin respuesta (4,6 s)	Completa (24,9 s): 12 errores de tool registrados, 3 reintentos (G3) y respuesta "esto NO significa que sea seguro"
Q3: nombres inválidos -> falso "limpio"	Validación de nombre por ecosistema en *query_osv*, que devuelve un error corregible, más una nota de que "0 hallazgos no significa seguro"	0 vulnerabilidades, analista x3, 16.854 tokens	Detecta pillow y pyyaml críticos; analista x1; 20.917 tokens
F2 / Q3: analista sobre datos vacíos y "no hace falta parchear"	Guardrail G7 (sin vulnerabilidades o fuera de alcance -> FINISH) y una síntesis que nunca declara seguro sin datos	40,0 s, 22.389 tokens, conclusión falsa	6,6 s, 3.973 tokens (-82%), declara "fuera de alcance"

La corrección de F2 necesitó dos iteraciones, y también se detectó en Phoenix: en *F2-out-of-scope-fix* el Investigador declaró "FUERA DE ALCANCE", pero el supervisor LLM lo volvió a enviar dos veces más (12,7 s). Se extendió G7 para que el fuera de alcance corte siempre, y en *F2-out-of-scope-fix2* terminó en 2 pasos (6,6 s). Todas las reglas nuevas tienen tests sin LLM (16 tests en verde).

7. Recomendaciones

- **Circuit breaker para herramientas caídas.** En F1-fix, los 3 reintentos del Investigador consumieron 23.240 tokens y 24,9 s sin chance de éxito. Si todas las llamadas de una ejecución fallan con error de conexión, conviene cortar en el primer intento.
- **Reducir la latencia percibida:** como la latencia depende de los tokens de salida, conviene hacer streaming de la síntesis al usuario y acortar el formato del Analista (la Síntesis ya reescribe la tabla: hoy se genera dos veces).
- **Reducir el costo del Investigador:** resumir los resultados de *query_osv* antes de devolverlos al LLM. El prompt crece de ~1.800 a ~10.000 tokens por los JSON de OSV, que ya se guardan aparte en *vulnerability_data*.
- **Detectar fallas silenciosas:** marcar como error los resultados con clave *error* (como el 401 de Tavily), para que aparezcan en rojo en Phoenix y no solo dentro del output.
- **Configurar el precio del modelo en Phoenix** para que el costo se vea en el dashboard, y agregar evaluaciones automáticas (p. ej. LLM-as-judge sobre la respuesta final) usando las trazas como dataset.

Archivos del análisis en el repositorio: *scripts/generate_traffic.py* (tráfico y fault injection), *scripts/analyze_traces.py* (métricas desde la API de Phoenix), *scripts/build_report.py* (este PDF) y *docs/observability/* (métricas, gráficos y capturas).